

Assignment 4: PennyWise — Personal Finance Tracker

Pending



Objective

Build a **FastAPI-based Personal Finance Tracker API** where users can record income and expenses, view summaries, filter transactions, update entries, and perform soft deletes.

The assignment should demonstrate:

- Global Exception Handlers
- OAuth2 Scopes
- Default Factories for timestamps
- User-specific transaction access
- Query Parameter Filtering
- Soft Delete
- Business Rule Validation

Technical Requirements

- **Framework:** FastAPI
 - **Authentication:** OAuth2 / JWT
 - OAuth2 scopes:
 - Read transactions
 - Write transactions
 - Strict mode must be stored for each user
 - `created_at` must be generated automatically using a default factory
 - Users should access only their own transactions
-

Data Models

User

- id
- email
- strict_mode
- scope

Transaction

- id
- user_id
- amount
- category
- description
- type
- created_at
- is_active

Transaction Type

Use an Enum for:

- income
- expense

API Routes

POST /transactions

Create a new income or expense transaction.

Required fields:

• Amount

- Amount
- Category
- Description
- Transaction type

Validation:

- Amount must not be 0
- Category is required
- Transaction type must be income or expense

If `strict_mode = true`, an expense must not be greater than the user's available balance.

GET /transactions/summary

Return the authenticated user's financial summary.

The response should include:

- Total income
- Total expense
- Current balance

Only active transactions should be considered.

GET /transactions

Return the authenticated user's transactions.

Support filtering using:

- month
- year

Example:

`/transactions?month=8&year=2026`

Only active transactions belonging to the current user should be returned.

PUT /transactions/{id}

Update an existing transaction.

Requirements:

- Users can update only their own transactions.
 - Validate amount, category, and transaction type.
 - The transaction date must **not be changed after 24 hours from creation**.
 - Return 404 Not Found if the transaction does not exist.
-

DELETE /transactions/{id}

Implement **soft delete**.

Do not permanently remove the transaction from the database.

Instead, update:

```
is_active = false
```

Deleted transactions should not appear in normal transaction listings or financial summaries.

Strict Mode Logic

Each user has a `strict_mode` setting.

If:

```
strict_mode = true
```

the system must prevent an expense when:

```
expense amount > current balance
```

Example:

If the current balance is ₹5,000 and the user attempts to create an expense of ₹6,000, the request must be rejected.

If strict mode is disabled, the transaction may be allowed.

OAuth2 Scopes

Implement OAuth2 scopes for transaction access.

Read Scope

Required for operations such as:

```
GET /transactions  
GET /transactions/summary
```

Write Scope

Required for operations such as:

```
POST /transactions  
PUT /transactions/{id}  
DELETE /transactions/{id}
```

Users without the required scope must not be allowed to access the respective endpoint.

Default Timestamp

The `created_at` value should be generated automatically.

Use a default factory instead of manually passing the timestamp for every transaction.

Example:

```
created_at: datetime = Field(default_factory=datetime.utcnow)
```

Global Exception Handlers

Create global exception handlers for common application errors.

Examples:

- Invalid transaction
- Insufficient balance

- Unauthorized access
- Missing permissions
- Transaction not found

Return meaningful error responses using appropriate HTTP status codes.

Validation Rules

- `amount` must not be `0`
 - `category` is mandatory
 - Transaction type must be `income` OR `expense`
 - Users can access only their own transactions
 - Strict mode must prevent expenses greater than balance
 - Soft-deleted transactions must not appear in active results
 - `created_at` must be generated automatically
-

Error Handling

Handle appropriate errors for:

- Invalid amount
- Missing category
- Insufficient balance
- Invalid transaction type
- Transaction not found
- Unauthorized access
- Missing OAuth2 scope
- Attempt to modify another user's transaction

Use appropriate HTTP status codes such as `400`, `401`, `403`, `404`, and `422`.

API Testing

All APIs should be testable through FastAPI Swagger:

/docs

Test:

- Income creation
 - Expense creation
 - Strict mode validation
 - Transaction summary
 - Month/year filtering
 - Transaction updates
 - 24-hour date restriction
 - Soft delete
 - OAuth2 read/write scopes
 - Global exception handling
-

Submission Instructions

Submit the **complete project as ONE ZIP file through the LMS.**

The ZIP file should include:

- Complete source code
- requirements.txt
- README.md
- All files required to run the project

ZIP File Name

StudentName_PennyWise.zip

Example:

RahulKumar_PennyWise.zip

Important

- Submit only **one ZIP file**.
 - Do not upload individual files separately.
 - Do not include `venv`, `.venv`, `__pycache__`, or unnecessary folders.
 - Test the application before submission.
 - Ensure the correct ZIP file is uploaded successfully to the LMS.
 - Submit before the deadline mentioned in the LMS.
-

Evaluation Criteria

The assignment will be evaluated based on:

- Correct API functionality
- OAuth2 scopes implementation
- Strict mode logic
- Transaction summary calculation
- Month/year filtering
- Default timestamp handling
- 24-hour update restriction
- Soft delete implementation
- Global exception handlers
- User-specific data access
- Validation and error handling
- Code quality and organization
- Successful project execution
- Proper ZIP submission

Expected Outcome

Upload Assignment